

Technische Requirements

Inleiding

In dit onderdeel werk ik de technische requirements uit voor de oplossing van Reemo. In de eerdere onderdelen heb ik eerst gekeken naar wat het systeem moet doen. Dat zijn de functionele requirements. Daarna heb ik gekeken naar hoe goed het systeem moet werken, zoals snelheid, veiligheid en gebruiksvriendelijkheid. Dat zijn de niet-functionele requirements. In dit onderdeel ga ik een stap verder. Hier leg ik uit hoe deze eisen technisch gerealiseerd kunnen worden.

Technische requirements zijn dus niet zomaar losse technische keuzes. Ze komen voort uit de analyse die eerder is gedaan. Eerst kijk je naar de wensen en problemen van de stakeholders. Daarna vertaal je die naar functionele en niet-functionele eisen. Pas daarna kijk je welke techniek nodig is om deze eisen waar te maken. Juist daarom is dit onderdeel belangrijk. Het laat zien dat de oplossing niet alleen logisch klinkt op papier, maar ook echt gebouwd kan worden.

Voor Reemo is dit relevant, omdat het bedrijf nu nog veel handmatig werkt. Informatie staat verspreid over verschillende bestanden en communicatiekanalen. Daardoor ontstaan fouten, vertraging en onduidelijkheid. Als je daarvoor een digitale oplossing wilt ontwerpen, moet je niet alleen weten wat het systeem moet kunnen, maar ook hoe je het technisch goed opzet. In dit onderdeel laat ik zien hoe ik die vertaalslag heb gemaakt.

Theoretisch kader

Binnen de literatuur over softwareontwikkeling wordt vaak onderscheid gemaakt tussen functionele, niet-functionele en technische requirements. Atlassian (2024) legt uit dat functionele requirements beschrijven wat een systeem moet doen. Dat gaat bijvoorbeeld om functies zoals uren registreren, contracten uploaden of een dashboard tonen. Niet-functionele requirements beschrijven hoe goed het systeem moet werken. Daarbij gaat het om zaken zoals performance, security, betrouwbaarheid en gebruiksvriendelijkheid.

Technische requirements zijn weer iets anders. AltexSoft (2024) beschrijft dat technische requirements de technische vertaling zijn van deze eisen. Ze laten zien welke technische keuzes nodig zijn om het systeem in de praktijk werkend te maken. Je kijkt dan bijvoorbeeld naar de database, de systeemstructuur, beveiliging en prestaties. IBM (2024) laat ook zien dat je bij systeemarchitectuur moet kijken naar meerdere technische onderdelen, zoals dataopslag, verwerking, toegang en schaalbaarheid. Daarom heb ik mijn technische requirements ook langs die onderdelen uitgewerkt.

Kort gezegd is het verschil als volgt. Functionele requirements gaan over wat het systeem moet doen. Niet-functionele requirements gaan over hoe goed het systeem moet werken. Technische requirements gaan over hoe je dat technisch mogelijk maakt. Dat verschil is belangrijk, omdat deze drie soorten eisen wel met elkaar samenhangen, maar niet hetzelfde zijn.

Koppeling met functionele en niet-functionele requirements

De technische requirements in dit project zijn niet uit het niets ontstaan. Ze zijn opgesteld op basis van de functionele en niet-functionele requirements die eerder al uit de analyse naar voren kwamen. Vanuit de functionele kant was bijvoorbeeld duidelijk dat het systeem uren moet kunnen registreren, omzet automatisch moet kunnen berekenen en per klant een dashboard moet kunnen tonen. Vanuit de niet-functionele kant kwam naar voren dat het systeem snel moet reageren, veilig met gegevens moet omgaan en voor gebruikers duidelijk en eenvoudig moet zijn.

Juist deze eisen sturen de technische keuzes. Als een systeem bijvoorbeeld uren en contracten met elkaar moet verbinden, dan heb je een manier nodig om data gestructureerd op te slaan. Als een dashboard snel moet laden, dan moet de techniek daar rekening mee houden. En als gevoelige informatie, zoals tarieven en contracten, in het systeem komt te staan, dan moet je ook nadenken over toegang en beveiliging. Op die manier vormen de functionele en niet-functionele requirements de basis voor de technische requirements.

Belangrijkste technische requirements

Op basis van de analyse en de theorie zijn in dit project de volgende technische requirements opgesteld.

- Het systeem moet gebruikmaken van een centrale database waarin klantgegevens, developers, contracten en urenregistraties gestructureerd worden opgeslagen.
- Het systeem moet een backend bevatten waarin de logica en berekeningen worden verwerkt.
- Het systeem moet een duidelijke frontend hebben waarin gebruikers hun gegevens kunnen invoeren en overzichten kunnen bekijken.
- Het systeem moet rolgebaseerde toegang ondersteunen, zodat niet iedere gebruiker alle informatie kan zien.
- Tot slot moet het systeem technisch zo zijn ingericht dat het snel reageert en later uitgebreid kan worden wanneer Reemo groeit.

Deze technische requirements volgen direct uit de eerdere analyse. Ze zijn dus niet gekozen omdat ze technisch interessant klinken, maar omdat ze nodig zijn om de functionele en niet-functionele eisen te ondersteunen.

Database en data-opslag

Een van de belangrijkste technische keuzes is de keuze voor een relationele database, zoals PostgreSQL. Een relationele database is een database waarin gegevens in tabellen worden opgeslagen en waarin deze tabellen met elkaar verbonden zijn. Dat betekent dat gegevens niet los van elkaar staan, maar in relatie met elkaar worden beheerd. In het geval van Reemo is dat belangrijk, omdat klanten, developers, contracten en urenregistraties allemaal met elkaar samenhangen.

Deze keuze komt voort uit zowel de praktijk als de theorie. In de huidige situatie van Reemo staat data verspreid over e-mails, PDF-bestanden, berichten en losse documenten. Daardoor is er geen centraal overzicht en is de kans op fouten groot. Vanuit de functionele requirements kwam daarom naar voren dat gegevens centraal opgeslagen moeten worden. Vanuit de niet-functionele requirements kwam daar nog bij dat de data betrouwbaar en consistent moet zijn. Volgens IBM (2024) is een goede data-architectuur een belangrijk onderdeel van systeemarchitectuur, omdat systemen anders niet stabiel en beheersbaar blijven.

Een relationele database past hier goed bij, omdat de gegevens van Reemo logisch aan elkaar gekoppeld moeten worden. Een urenregistratie hoort bijvoorbeeld bij een developer, maar ook bij een contract. Een contract hoort weer bij een klant. Door gebruik te maken van relaties in een database blijven die koppelingen duidelijk en correct. Dat maakt het ook mogelijk om later automatisch berekeningen te doen, zoals het berekenen van omzet op basis van uren en tarieven. PostgreSQL is hierbij een logische keuze, omdat het een veelgebruikte en betrouwbare relationele database is die goed past bij gestructureerde bedrijfsdata.

Backend en systeemstructuur

Een tweede belangrijke technische requirement is dat het systeem een backend moet hebben waarin de belangrijkste logica wordt verwerkt. De backend is het deel van de applicatie dat op de achtergrond werkt. Hier worden gegevens verwerkt, berekeningen uitgevoerd en data opgehaald of opgeslagen in de database.

Deze keuze hangt direct samen met de requirements van het systeem. Een van de functionele eisen is bijvoorbeeld dat omzet automatisch berekend moet worden op basis van uren en tarieven. Dat soort berekeningen wil je niet afhankelijk maken van de gebruiker of alleen in de interface uitvoeren. Volgens AltexSoft (2024) is het in softwareontwikkeling gebruikelijk om de logica van een systeem los te houden van de interface. Daardoor blijft het systeem beter beheersbaar en worden fouten beperkt.

Voor Reemo betekent dit dat de backend verantwoordelijk wordt voor taken zoals het verwerken van urenregistraties, het ophalen van contractgegevens en het uitvoeren van omzetberekeningen. Dit maakt het systeem betrouwbaarder en overzichtelijker. Ook sluit dit aan op de niet-functionele eis dat het systeem correct en stabiel moet werken.

Frontend en gebruikersinterface

Een andere technische requirement is dat het systeem een duidelijke frontend moet hebben. De frontend is het deel waarmee gebruikers direct werken. Hier voert een developer bijvoorbeeld uren in, en hier bekijkt een manager of administratie een dashboard of een overzicht.

Deze keuze komt vooral voort uit de niet-functionele requirement dat het systeem gebruiksvriendelijk moet zijn. Atlassian (2024) laat zien dat niet-functionele requirements zoals usability belangrijk zijn, omdat een systeem anders wel kan werken, maar in de praktijk toch slecht gebruikt wordt. Voor Reemo is dat belangrijk, omdat het systeem juist bedoeld is om handmatig werk te verminderen en meer overzicht te geven. Als de interface onduidelijk of ingewikkeld is, lost het systeem het probleem dus niet echt op.

Daarom moet de frontend niet alleen netjes zijn, maar vooral logisch en duidelijk. Gebruikers moeten snel begrijpen waar ze iets kunnen invoeren of terugvinden. Deze technische requirement sluit ook goed aan op de wireframes die later worden gemaakt. Daarin wordt zichtbaar hoe de oplossing er in een eenvoudige vorm uit kan zien.

Beveiliging en toegangsbeheer

Beveiliging is ook een belangrijk onderdeel van de technische requirements. In het systeem komen gevoelige gegevens te staan, zoals klantinformatie, contracten en tarieven. Daarom is het niet genoeg dat data alleen wordt opgeslagen. Er moet ook goed worden nagedacht over wie welke gegevens mag zien en gebruiken.

Volgens IBM (2024) hoort toegangsbeheer bij een goede systeemarchitectuur, zeker wanneer een systeem met gevoelige of bedrijfskritische gegevens werkt. Daarom is in dit project gekozen voor rolgebaseerde toegang. Dit betekent dat gebruikers alleen toegang hebben tot de informatie die bij hun rol hoort. Een developer hoeft bijvoorbeeld niet alle contracten of tarieven te kunnen zien, terwijl een administratieve gebruiker of manager meer overzicht nodig heeft.

Deze technische requirement komt direct voort uit de niet-functionele requirement dat het systeem veilig moet zijn. Tegelijk sluit het ook aan op de praktijk van Reemo, omdat verschillende gebruikers in het proces een andere rol hebben. Door rechten te koppelen aan rollen blijft het systeem overzichtelijker en wordt de kans kleiner dat gegevens verkeerd worden ingezien of aangepast.

Performance en schaalbaarheid

Tot slot zijn ook performance en schaalbaarheid als technische requirements meegenomen.

Performance gaat over hoe snel en efficiënt het systeem werkt. Schaalbaarheid gaat over de vraag of het systeem later ook nog goed werkt als het bedrijf groeit of als er meer gebruikers bijkomen.

Deze keuze komt voort uit de niet-functionele requirements rondom snelheid en betrouwbaarheid.

Een dashboard heeft weinig waarde als het langzaam laadt of vastloopt. Daarnaast is het voor Reemo belangrijk dat de oplossing niet alleen nu werkt, maar ook later nog bruikbaar blijft. AltexSoft (2024) benoemt dat technische keuzes in softwareontwikkeling ook rekening moeten houden met toekomstig gebruik, zodat een systeem niet direct opnieuw ontworpen hoeft te worden wanneer het groeit.

Voor dit project betekent dit dat de database efficiënt moet kunnen omgaan met data en dat de systeemstructuur overzichtelijk en uitbreidbaar moet blijven. Daardoor ontstaat een oplossing die niet alleen voor de huidige situatie bruikbaar is, maar ook toekomstbestendig is.

Hoe de keuzes zijn gemaakt

De technische keuzes in dit document zijn dus niet willekeurig gemaakt. Eerst is gekeken naar de problemen in de huidige situatie van Reemo. Daarna zijn op basis daarvan functionele en niet-functionele requirements opgesteld. Vervolgens is met behulp van theorie en technisch onderzoek bepaald welke technische onderdelen nodig zijn om aan deze eisen te voldoen.

Daarom is bijvoorbeeld gekozen voor een relationele database, omdat de gegevens van Reemo sterk met elkaar samenhangen en betrouwbaar opgeslagen moeten worden. Daarom is ook gekozen voor een backend, omdat berekeningen en verwerkingen centraal en correct moeten plaatsvinden. En daarom is er gekeken naar beveiliging, performance en schaalbaarheid, omdat deze punten in de theorie steeds terugkomen als belangrijke onderdelen van technische requirements en systeemarchitectuur.

Conclusie

De technische requirements vormen de brug tussen de analyse en de uiteindelijke oplossing. Waar de functionele requirements beschrijven wat het systeem moet doen en de niet-functionele requirements beschrijven hoe goed het systeem moet werken, laten de technische requirements zien hoe dit technisch mogelijk wordt gemaakt.

In dit project zijn de technische requirements opgesteld op basis van de analyse van Reemo, de eerder geformuleerde functionele en niet-functionele eisen en relevante theorie uit softwareontwikkeling en systeemarchitectuur. Daardoor ontstaat een oplossing die niet alleen logisch is vanuit de praktijk, maar ook technisch goed onderbouwd en uitvoerbaar is.

Bronnen (APA-stijl)

AltexSoft. (2024). Functional vs technical requirements in software development. Geraadpleegd van <https://www.altexsoft.com>

Atlassian. (2024). Functional vs non-functional requirements. Geraadpleegd van <https://www.atlassian.com>

IBM. (2024). What is system architecture? Geraadpleegd van <https://www.ibm.com>